

Junior Software Engineer Technical Assessment

Order Processing API

Objective

Build a small backend application that allows customers to create and manage orders.

You may use any programming language or framework you are comfortable with.

AI & External Tools

You are allowed to use AI tools such as ChatGPT, Claude, GitHub Copilot, Cursor, or similar tools.

You may also use documentation, Google, Stack Overflow, and any other resources you would normally use while developing software.

There is no penalty for using AI.

However, you should understand the code you submit and be prepared to explain your implementation and technical decisions.

Scenario

You are building part of an order management system.

Customers should be able to create orders containing one or more products.

An example request might look like:

```
{
  "customer_id": 123,
  "items": [
    {
      "product_id": 10,
```

```
    "quantity": 3,  
    "unit_price": 250  
  },  
  {  
    "product_id": 15,  
    "quantity": 2,  
    "unit_price": 100  
  }  
]  
}
```

You are responsible for deciding how the application, database, models, services, endpoints, and other components should be structured.

Functional Requirements

1. Create an Order

A customer should be able to create an order containing one or more items.

Each item contains:

- Product ID
- Quantity
- Unit price

The backend must calculate the subtotal, applicable discount, and final total.

A total or discount supplied by the client should not be trusted.

2. Retrieve an Order

It should be possible to retrieve an individual order and its items.

The response should include enough information to understand how the final amount was calculated.

For example:

```
{
  "id": 1,
  "customer_id": 123,
  "status": "pending",
  "subtotal": 12000,
  "discount": 600,
  "discount_percentage": 5,
  "total": 11400,
  "items": []
}
```

3. List Orders

It should be possible to retrieve a list of orders.

4. Update Order Status

An order can have one of the following statuses:

- pending
- processing
- completed
- cancelled

Only the following transitions are allowed:

pending -> processing
pending -> cancelled

processing -> completed
processing -> cancelled

Other status transitions should not be allowed.

Discounts

The system supports automatic discounts based on the value of an order.

The following rules apply:

Subtotal $\leq$ 5,000

No discount

Subtotal $>$ 5,000 and $\leq$ 10,000

2% discount

Subtotal $>$ 10,000 and $\leq$ 20,000

5% discount

Subtotal $>$ 20,000

10% discount

For example:

Order A

Subtotal: 4,500

Discount: 0

Total: 4,500

Order B

Subtotal: 8,000

Discount: 160 (2%)

Total: 7,840

Order C

Subtotal: 12,000

Discount: 600 (5%)

Total: 11,400

Order D

Subtotal: 25,000

Discount: 2,500 (10%)

Total: 22,500

The discount must be determined and calculated by the backend.

The client should not be able to choose its own discount percentage or discount amount.

The application should retain enough information to determine:

- Original subtotal
 - Discount percentage applied
 - Discount amount
 - Final total
-

Business Rules

1. An order must contain at least one item.
2. Item quantity must be greater than zero.
3. Unit prices cannot be negative.
4. The subtotal must be calculated by the backend.
5. Discounts must be calculated by the backend according to the discount rules.
6. The final total should be:

$\text{subtotal} - \text{discount} = \text{total}$

7. Completed orders cannot be modified.
8. Cancelled orders cannot be modified.
9. Creating an order and all of its items must behave as a single operation. The system should not leave partially created orders if something goes wrong.
10. The API should protect against accidentally creating the same order multiple times when the same request is retried.

For example, imagine a client sends an order creation request but loses its connection before receiving the response.

The client retries exactly the same request.

Your application should have a reasonable mechanism for preventing this from creating two orders.

You may decide how to implement this.

Validation & Error Handling

The API should return appropriate errors for invalid requests.

Examples include:

No order items

Quantity ≤ 0

Negative price

Invalid order status

Invalid status transition

Attempting to modify a completed order

Attempting to modify a cancelled order

Invalid or malformed request data

Error responses should be understandable to someone consuming the API.

Testing

Include automated tests for the parts of the application you consider important.

At minimum, important business rules should be covered.

Discount boundary conditions should also be considered.

For example, consider what should happen when the subtotal is exactly:

5,000

5,001

10,000

10,001

20,000

20,001

You may choose your testing strategy and tools.

README

Include a README containing:

- How to install the application
 - How to run it
 - How to configure the database
 - How to run the tests
 - API endpoints implemented
 - Assumptions made
 - Important technical decisions
 - Anything you would improve if you had more time
-

Submission

Submit the complete project through the agreed submission method.

The project should run using the instructions in your README.

Do not worry if you cannot implement everything perfectly.

We are interested in the quality of your reasoning and implementation, not simply the number of requirements completed.